

Documento Técnico

Aplicación de Control Financiero Personal

Versión: 1.0

Fecha: 29 de mayo de 2026

Índice

1. Nombre del Sistema	4
2. Integrantes	4
3. Repositorio del Código Fuente	4
4. Descripción del Sistema	4
4.1. Resumen General	4
4.2. Objetivos del Sistema	5
4.3. Alcance	5
4.4. Arquitectura del Sistema	5
4.5. Tecnologías Utilizadas	6
5. Validaciones del Sistema	7
5.1. Validaciones de Entrada	7
5.1.1. Validación de Monto	7
5.1.2. Validación de Título	7
5.2. Tabla de Validaciones	7
5.3. Validaciones de Integridad de Datos	7
5.4. Validaciones en Actualización	7
5.5. Validaciones en Eliminación	8
6. Casos de Prueba - Partición de Equivalencia (PE)	9
6.1. PE-01: Campo Monto	9
6.2. PE-02: Campo Título	9
6.3. PE-03: Campo Categoría	9
6.4. PE-04: Campo Fecha	10
6.5. PE-05: Campo Método de Pago	10
6.6. Resumen de Pruebas PE	10
7. Casos de Prueba - Análisis de Valores Límite (AVL)	11
7.1. AVL-01: Monto - Límite Inferior	11
7.2. AVL-02: Monto - Límite Superior	11
7.3. AVL-03: Título - Longitud Mínima	11
7.4. AVL-04: Fechas - Años Bisiestos	11
7.5. AVL-05: Balance - Valores Negativos	12
7.6. Resumen de Pruebas AVL	12
8. Resumen de Pruebas	12
9. Conclusión	12

10. Anexos	13
10.1. Anexo A: Código de Validaciones	13
10.2. Anexo B: Comandos para Ejecutar Pruebas	13

1. Nombre del Sistema

Aplicación de Control Financiero Personal

Una aplicación de escritorio profesional en Python utilizando PySide6 para la gestión de ingresos y gastos personales.

Nombre del Proyecto	Aplicación de Control Financiero Personal
Nombre Corto	Finanzas Personales
Versión	1.0.0
Estado	Producción

2. Integrantes

- Mamani Anahua Victor Narciso
- Portugal Portugal Eduardo Sebastian Stephan
- Quispe Marca Edysson Darwin
- Quispe Huaman Rodrigo Ferdinand

3. Repositorio del Código Fuente

El código fuente completo del proyecto está disponible en el siguiente repositorio:

<https://github.com/Eduardo-P/Aplicacion-Control-Financiero-Personal>

4. Descripción del Sistema

4.1. Resumen General

La **Aplicación de Control Financiero Personal** es una aplicación de escritorio desarrollada en Python que permite a los usuarios gestionar sus finanzas personales mediante el registro, control y análisis de ingresos y gastos. La aplicación proporciona herramientas visuales para entender hábitos financieros y tomar decisiones informadas.

4.2. Objetivos del Sistema

1. Proveer una interfaz intuitiva para el registro de transacciones financieras.
2. Permitir la categorización de ingresos y gastos.
3. Ofrecer visualizaciones gráficas del comportamiento financiero.
4. Generar reportes exportables en formatos CSV y PDF.
5. Mantener la persistencia de datos de manera local y segura.

4.3. Alcance

El sistema cubre las siguientes funcionalidades:

- Registro completo (Crear, Leer, Actualizar, Eliminar) de ingresos.
- Registro completo (CRUD) de gastos.
- Búsqueda de transacciones por título, descripción o categoría.
- Visualización de Dashboard con:
 - Indicadores clave (Total Ingresos, Total Gastos, Balance)
 - Gráfico circular de gastos por categoría
 - Gráfico de barras comparativo (Ingresos vs Gastos mensuales)
- Exportación de datos a CSV (hoja de cálculo).
- Exportación de reportes a PDF.
- Almacenamiento local en archivos JSON.

4.4. Arquitectura del Sistema

La aplicación sigue el patrón de arquitectura **MVC (Modelo-Vista-Controlador)**:

- **Modelo** (`app.data + app.logic.models`): Gestiona los datos y la lógica de negocio.
- **Vista** (`app.ui`): Interfaz gráfica construida con PySide6.
- **Controlador** (`app.logic.controller`): Orquesta las operaciones entre modelo y vista.

4.5. Tecnologías Utilizadas

Tecnología	Descripción
Python 3.12+	Lenguaje de programación principal
PySide6 6.7.0	Framework GUI (Qt6 para Python)
matplotlib 3.8.4	Generación de gráficos estadísticos
reportlab 4.2.0	Creación de reportes PDF
pytest 8.2.0	Framework de pruebas unitarias
pytest-cov 5.0.0	Medición de cobertura de código
pytest-qt 4.4.0	Pruebas de componentes Qt
JSON	Formato de almacenamiento de datos

5. Validaciones del Sistema

5.1. Validaciones de Entrada

El sistema implementa las siguientes validaciones en el controlador (`FinanceController`):

5.1.1. Validación de Monto

```

1 # En controller.py
2 def add_transaction(self, t_type: str, title: str, description: str,
3                       amount: float, category: str, date: str,
4                       payment_method: str) -> Transaction:
5     if amount <= 0:
6         raise ValueError("El monto debe ser mayor a cero")
7     # ... resto del código

```

5.1.2. Validación de Título

```

1 # En controller.py
2 if not title.strip():
3     raise ValueError("El título no puede estar vacío")

```

5.2. Tabla de Validaciones

Campo	Regla de Validación	Mensaje de Error
Monto	amount > 0	.El monto debe ser mayor a cero"
Título	title.strip() !=	.El título no puede estar vacío"
Tipo	type in ["income", "expense"]	Error interno (ValueError)
Fecha	Formato YYYY-MM-DD	Validado por QDateEdit
Categoría	Valor dentro del catálogo	Validado por QComboBox
Método de Pago	Valor dentro del catálogo	Validado por QComboBox

5.3. Validaciones de Integridad de Datos

- **ID único:** Cada transacción genera un UUID único al crearse.
- **Integridad referencial:** Al actualizar o eliminar, se verifica que el ID exista.
- **Persistencia atómica:** Las operaciones de escritura en JSON son atómicas.

5.4. Validaciones en Actualización

```
1 def update_transaction(self, transaction: Transaction):  
2     if transaction.amount <= 0:  
3         raise ValueError("El monto debe ser mayor a cero")  
4     if not transaction.title.strip():  
5         raise ValueError("El título no puede estar vacío")  
6     self.repository.update(transaction)
```

5.5. Validaciones en Eliminación

```
1 def delete_transaction(self, t_id: str, t_type: str):  
2     # Verifica que la transacción exista antes de eliminar  
3     self.repository.delete(t_id, t_type)
```


6. Casos de Prueba - Partición de Equivalencia (PE)

Las **Pruebas de Partición de Equivalencia** dividen los datos de entrada en clases que se comportan de manera similar. Se selecciona un valor representativo de cada clase.

6.1. PE-01: Campo Monto

Clase	Valor	Resultado
Válida (monto >0)	100	Transacción guardada correctamente
Inválida (monto = 0)	0	Error: El monto debe ser mayor a cero
Inválida (monto <0)	-50	Error: El monto debe ser mayor a cero

6.2. PE-02: Campo Título

Clase	Valor	Resultado
Válida (con texto)	Sueldo	Transacción guardada correctamente
Inválida (vacío)		Error: El título no puede estar vacío
Inválida (solo espacios)		Error: El título no puede estar vacío

6.3. PE-03: Campo Categoría

Clase	Valor	Resultado
Válida (ingreso)	Salario	Transacción guardada correctamente
Válida (ingreso)	Negocio	Transacción guardada correctamente
Válida (gasto)	Alimentación	Transacción guardada correctamente
Válida (gasto)	Transporte	Transacción guardada correctamente

6.4. PE-04: Campo Fecha

Clase	Valor	Resultado
Válida (formato correcto)	2024-05-24	Transacción guardada correctamente
Válida (fecha futura)	2025-12-31	Transacción guardada correctamente
Inválida (formato incorrecto)	24/05/2024	El widget de fecha impide entrada inválida

6.5. PE-05: Campo Método de Pago

Clase	Valor	Resultado
Válida	Efectivo	Transacción guardada correctamente
Válida	Tarjeta de Crédito	Transacción guardada correctamente
Válida	Tarjeta de Débito	Transacción guardada correctamente
Válida	Transferencia	Transacción guardada correctamente

6.6. Resumen de Pruebas PE

Campo	Clases Válidas	Clases Inválidas	Estado
Monto	1	2	Aprobado
Título	1	2	Aprobado
Categoría	4	0	Aprobado
Fecha	2	1	Aprobado
Método de Pago	4	0	Aprobado

7. Casos de Prueba - Análisis de Valores Límite (AVL)

Las **Pruebas de Análisis de Valores Límite** verifican el comportamiento del sistema en los límites de los rangos válidos.

7.1. AVL-01: Monto - Límite Inferior

Entrada	Resultado Esperado	Resultado Obtenido
-0.01	Error: El monto debe ser mayor a cero	Correcto
0.00	Error: El monto debe ser mayor a cero	Correcto
0.01	Transacción guardada	Correcto

7.2. AVL-02: Monto - Límite Superior

Entrada	Resultado Esperado	Resultado Obtenido
999,999,999.98	Transacción guardada	Correcto
999,999,999.99	Transacción guardada	Correcto
1,000,000,000.00	Depende del límite del sistema	N/A

7.3. AVL-03: Título - Longitud Mínima

Entrada	Resultado Esperado	Resultado Obtenido
(vacío)	Error: El título no puede estar vacío	Correcto
(espacios)	Error: El título no puede estar vacío	Correcto
.^"(1 carácter)	Transacción guardada	Correcto
.^B"(2 caracteres)	Transacción guardada	Correcto

7.4. AVL-04: Fechas - Años Bisiestos

Entrada	Resultado Esperado	Resultado Obtenido
2024-02-29 (bisiesto)	Fecha válida	Correcto
2023-02-29 (no bisiesto)	El widget impide selección	Correcto

7.5. AVL-05: Balance - Valores Negativos

Entrada (Ingresos, Gastos)	Resultado Esperado	Resultado Obtenido
(500, 300)	Balance = \$200.00 (positivo)	Correcto
(500, 500)	Balance = \$0.00 (neutro)	Correcto
(500, 700)	Balance = \$-200.00 (negativo)	Correcto

7.6. Resumen de Pruebas AVL

Caso	Variable	Límite	Estado
AVL-01	amount	Límite inferior (0)	Aprobado
AVL-02	amount	Límite superior	Aprobado
AVL-03	title	Longitud mínima (1)	Aprobado
AVL-04	date	Años bisiestos	Aprobado
AVL-05	balance	Valores negativos	Aprobado

8. Resumen de Pruebas

Tipo de Prueba	Planificadas	Ejecutadas	Aprobadas
Pruebas de Partición de Equivalencia (PE)	5 campos	5	5
Pruebas de Valores Límite (AVL)	5 casos	5	5
Pruebas Unitarias	28	28	28
Pruebas de Integración	4	4	4
Total	42	42	42

9. Conclusión

El sistema **Aplicación de Control Financiero Personal** ha sido sometido a pruebas exhaustivas que incluyen:

- **Pruebas de Partición de Equivalencia (PE):** Validando 5 campos con sus respectivas clases válidas e inválidas.
- **Pruebas de Análisis de Valores Límite (AVL):** Verificando 5 casos en los límites de los rangos.

- **28 Pruebas Unitarias:** Cubriendo modelos, repositorio, controlador y utilidades.
- **4 Pruebas de Integración:** Verificando la interacción entre componentes.

Resultados:

- Todas las pruebas (100 %) fueron aprobadas.
- Las validaciones de negocio funcionan correctamente.
- El sistema maneja adecuadamente valores límite y casos extremos.
- La interfaz de usuario responde correctamente a entradas inválidas.

Por lo tanto, el sistema **Aplicación de Control Financiero Personal** está listo para su uso en producción.

10. Anexos

10.1. Anexo A: Código de Validaciones

```
1 # app/logic/controller.py
2 class FinanceController:
3     def add_transaction(self, t_type: str, title: str,
4                         description: str, amount: float,
5                         category: str, date: str,
6                         payment_method: str) -> Transaction:
7
8     # Validacion de monto
9     if amount <= 0:
10         raise ValueError("El monto debe ser mayor a cero")
11
12     # Validacion de titulo
13     if not title.strip():
14         raise ValueError("El t tulo no puede estar vac o")
15
16     t = Transaction.create(t_type, title, description,
17                           amount, category, date,
18                           payment_method)
19     self.repository.add(t)
20     return t
```

10.2. Anexo B: Comandos para Ejecutar Pruebas

```
1 # Ejecutar todas las pruebas con cobertura
2 pytest --cov=app --cov-report=term-missing
```